GUION TÉCNICO DE DEFENSA AUDIOVISUAL

Prueba Diagnóstica (10%) — Lenguajes y Compiladores (Lapso 2026-II)

UNEG — Puerto Ordaz
Prof. Msc. Félix Márquez

Equipo: ParseCore

Eslogan: "*Estructurando la sintaxis, optimizando la ejecución.*"

Ponente: Brayan Carreño (C.I. V-32.015.527)

Sección: 01 | **Duración Máxima:** 5:00 min (Objetivo: 4:30 min)

Bloque 1: Presentación Formal Obligatoria

0:00 - 0:25 (25 seg)

Acción en Pantalla: Cámara web activa en primer plano o esquina. Mostrar en pantalla la diapositiva de portada del equipo ParseCore.

> "*Saludos cordiales, profesor Félix Márquez.*
> *Somos el grupo **ParseCore**, bajo el eslogan: '**Estructurando la sintaxis, optimizando la ejecución.**'*
> *Mi nombre es **Brayan Carreño**, portador de la cédula de identidad **V-32.015.527**, cursante de la sección **01** en la cátedra de **Lenguajes y Compiladores**.*
> *A continuación, presento la defensa técnica y demostración en vivo de los tres ejercicios desarrollados para la Prueba Diagnóstica.*"

Bloque 2: Ejercicio 1 — Movimientos de Caballo en Ajedrez 8x8

0:25 - 1:40 (1 min 15 seg)

> **Acción en Pantalla:** Abrir VS Code en `Ejercicio_1_Ajedrez_Caballo`. Ejecutar en terminal: `.\caballo.exe`.

*"Iniciamos con el **Ejercicio 1**, cuyo objetivo es validar una secuencia algebraica de movimientos de un caballo en un tablero de 8x8.*
*Matemáticamente, el caballo se desplaza en forma de L, lo cual modelamos formalmente mediante la relación cuadrática **$\Delta x^2 + \Delta y^2 = 5$**, equivalente a que el producto de las diferencias de coordenadas sea 2 y su suma sea 3.*

Como observamos en la terminal, nuestro algoritmo valida en cortocircuito el prefijo 'K'; si se detecta una pieza no permitida como 'Q', reporta de inmediato: 'movimiento INCORRECTO falla letra Q no pertenece a un caballo'. Si el salto no es en L o sale del tablero, identifica con exactitud el número del movimiento erróneo, superando al 100% los casos de prueba del enunciado.

*En cuanto al rendimiento, ejecutamos 10 iteraciones de 500 movimientos: en **Lenguaje C** toma apenas **0.12 milisegundos** en total. Y en el benchmark de **n = 100 movimientos**, **C ejecuta en solo 2.39 microsegundos** frente a **36.7 microsegundos en Python**, alcanzando una aceleración de **15.3 veces más rápido en C** gracias al acceso contiguo a memoria y la optimización en registros de CPU."*

Bloque 3: Ejercicio 2 — Polinomio (x+1)^n y Pascal Dinámico

1:40 - 2:50 (1 min 10 seg)

> **Acción en Pantalla:** Abrir `pascal_polinomio.c` en VS Code. Ejecutar en terminal: `.\pascal_polinomio.exe`.

> *"Continuamos con el **Ejercicio 2**: generación de los coeficientes del polinomio $(x+1)^n$ mediante el Triángulo de Pascal y su evaluación numérica paso a paso.*
> *La consigna exige expresamente el uso de **memoria dinámica**. En nuestra implementación en C, cada nivel se reserva dinámicamente en el Heap con `malloc`, se calcula sumando los términos contiguos del nivel anterior, y se libera de inmediato la fila previa con `free`, garantizando cero fugas de memoria.*
>
> *Además, resolvimos un desafío crítico: para **n = 100**, el coeficiente central supera los **10^{29} (29 dígitos decimales)**, desbordando los 64 bits en C. Implementamos una estructura **BigInt** de alta precisión que calcula el valor exacto de 100 quintillones sin ninguna pérdida de precisión.*
>
> *Al evaluar con x = 2.0 para n = 4, vemos en pantalla el desglose término a término totalizando exactamente **81.0**, coincidiendo con $(2+1)^4$. En la prueba comparativa de n = 100, C genera la fila y gestiona la memoria en **184 microsegundos**."*

Bloque 4: Ejercicio 3 — Analizador Léxico (Traductor de C a Español) 2:50 - 4:10 (1 min 20 seg)

> **Acción en Pantalla:** Abrir `lexer_traductor.c` y `programa_prueba.c`. Ejecutar en terminal: `.\lexer_traductor.exe`. Abrir `programa_traducido.c` en vista dividida.

> *"Llegamos al **Ejercicio 3**, fundamento de nuestra materia: la construcción de un **Analizador Léxico** que carga un programa en C dinámicamente en memoria RAM y traduce sus palabras reservadas al español.*
> *Primero, medimos el archivo fuente con `fseek` y `ftell` y lo alojamos en el Heap con `malloc`. Luego, implementamos un autómata finito determinista que escanea el búfer caracter a caracter.*
>
> *Es esencial destacar las reglas de compilación aplicadas:*
> *1. **Cadenas de texto:** Si dentro de un literal aparece la palabra 'if' o 'while', se preserva intacta porque pertenece a un string.*
> *2. **Comentarios:** Los comentarios de una línea o multilínea se mantienen inalterados.*
> *3. **Identificadores:** Variables como 'contador_while' no se rompen porque representan tokens completos.*
>
> *Al ejecutar sobre nuestro archivo de prueba de 837 bytes, vemos en la vista dividida que 'int' se convierte en 'entero', 'if' en 'si', 'while' en 'mientras' y 'return' en 'retornar', emitiendo un reporte estadístico con la frecuencia de cada token reconocido."*

Bloque 5: Conclusiones Técnicas y Despedida 4:10 - 4:35 (25 seg)

> **Acción en Pantalla:** Mostrar en el navegador el repositorio GitHub: `github.com/Bryd3n/lyc-parsecore`.

> *"En conclusión, hemos implementado soluciones rigurosas cumpliendo con la gestión de memoria dinámica, la prevención de desbordamientos y el análisis sintáctico-léxico formal. Todo el código fuente, benchmarks y scripts de ejecución automatizada se encuentran versionados en el repositorio de nuestro equipo **ParseCore**.*
> *Muchísimas gracias por su atención, profesor Félix Márquez. Quedo atento a su evaluación."*

✅ Checklist Técnico Obligatorio Antes de Entregar

☐ **Duración del Video:** Cronometrado en ~4:30 min (NUNCA exceder los 5:00 minutos).

☐ **Formato:** Horizontal (16:9) obligatorio (Prohibido vertical / reels / shorts).

☐ **Cámara Web:** Tu rostro debe ser claramente visible durante toda la grabación.

☐ **Audio y Pantalla:** Audio nítido y código en VS Code con buen nivel de zoom (`Ctrl + +`).

☐ **Google Drive:** Subir el video con acceso público en modo: "*Cualquier persona con el enlace puede ver*".

☐ **video_defensa.txt:** Pegar el enlace en `lyc-parsecore/PRUEBA DIAGNOSTICA/Brayan_Carreno/video_defensa.txt` y hacer `git push` .